When Does Graph Compilation Help? Workload-Aware Optimization of Transformer Inference

Navya Kumar (nk4276) Fahad Tajiki (fxt2021)
Demetri López (dsl418) Tanish Vardhineni (tv2291)
Efficient AI and Hardware Accelerators — Project Proposal

I. Why This Project Matters

Running an AI model is expensive, especially when many requests must be served quickly. Compilers such as Torch-TensorRT can speed up PyTorch models by turning supported operations into optimized GPU engines. However, some operations remain in PyTorch. The model may then execute as a series of TensorRT and PyTorch regions instead of one continuous compiled program. These boundaries can introduce framework dispatch and extra GPU launches. Compiling more operations is therefore not always faster: a collection of tiny engines may cost more to execute than it saves [1], [2]. The actual cost depends on the model and the size of its inputs.

II. Question and Proposed Approach

Research question: *When do boundaries between TensorRT and PyTorch reduce inference speed, and can we choose a better compilation configuration for each workload?* We will use a small, pretrained Transformer in inference mode (no training), examine its compilation graph, and measure latency across input batch sizes and sequence lengths. We will then build a lightweight method that selects among *existing* compiler configurations using their measured or estimated costs. We are not building a new compiler; the contribution is a careful study of graph-break overhead and a practical configuration-selection method. We hypothesize that the best setting varies across workloads.

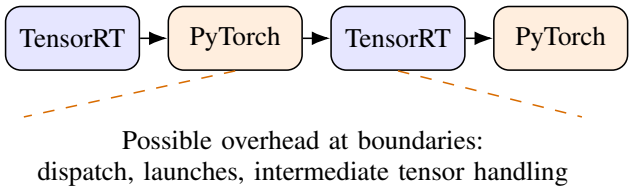


Fig. 1. A fragmented computation graph. Boundaries can incur overhead, but actual costs must be measured; they do not necessarily involve GPU–CPU transfers.

III. Project Plan and Individual Experiments

All four experiments will use the same model, GPU, software versions, and repeatable benchmark harness. We will first measure PyTorch and default Torch-TensorRT, then perform the following linked experiments:

TABLE I
Four experiments and individual ownership

Experiment	Owner	Main deliverable
E1: Graph partitioning	Navya	Partition maps and latency vs. compiler settings
E2: Workload sensitivity	Fahad	Heatmap of best settings by batch and sequence length
E3: GPU bottleneck analysis	Demetri	Nsight traces explaining launch/dispatch overhead
E4: Workload-aware selection	Tanish	Cost-based selector and speedup/tuning-cost results

E1: Vary minimum compiled region size and available partitioning strategies. Record TensorRT regions, fallback locations, and end-to-end latency. **E2:** Compare shared configurations for batch sizes 1, 4, 8, 16 and sequence lengths 128–1024 (as GPU memory permits), holding model weights and precision constant. **E3:** Profile representative runs using PyTorch Profiler and Nsight Systems. Contrast naturally occurring fallback regions with deliberately controlled graph breaks; keep their conclusions separate. **E4:** Use part of the benchmark results to build a simple lookup-table or regression-based selector, then test it on held-out workload shapes.

IV. Evaluation and Expected Outcome

We will compare PyTorch, default Torch-TensorRT, one fixed tuned configuration, our workload-aware selector, and the best tested configuration as an upper benchmark. We will report warmed-up median and p95 latency, throughput, GPU memory, numerical agreement, partition counts, and tuning time. Tests will use identical inputs and FP16 precision where supported. Our result will show *whether* graph breaks are a measurable bottleneck, *when* compiler settings should change, and whether smarter selection is worthwhile. If no meaningful speedup occurs, we will explain why using the profiler evidence.

References

[1] PyTorch, "Torch-TensorRT: Partitioning Phase," https://docs.pytorch.org/TensorRT/contributors/partitioning.html.
[2] PyTorch, "Torch-TensorRT: Performance Tuning Guide," https://docs.pytorch.org/TensorRT/user_guide/performance_tuning.html.